Lab 3 — Architectural Justification

Internal Microservice Catalog & Health Portal

Name : Likith Kumar K M
SRN : PES1UG24AM373

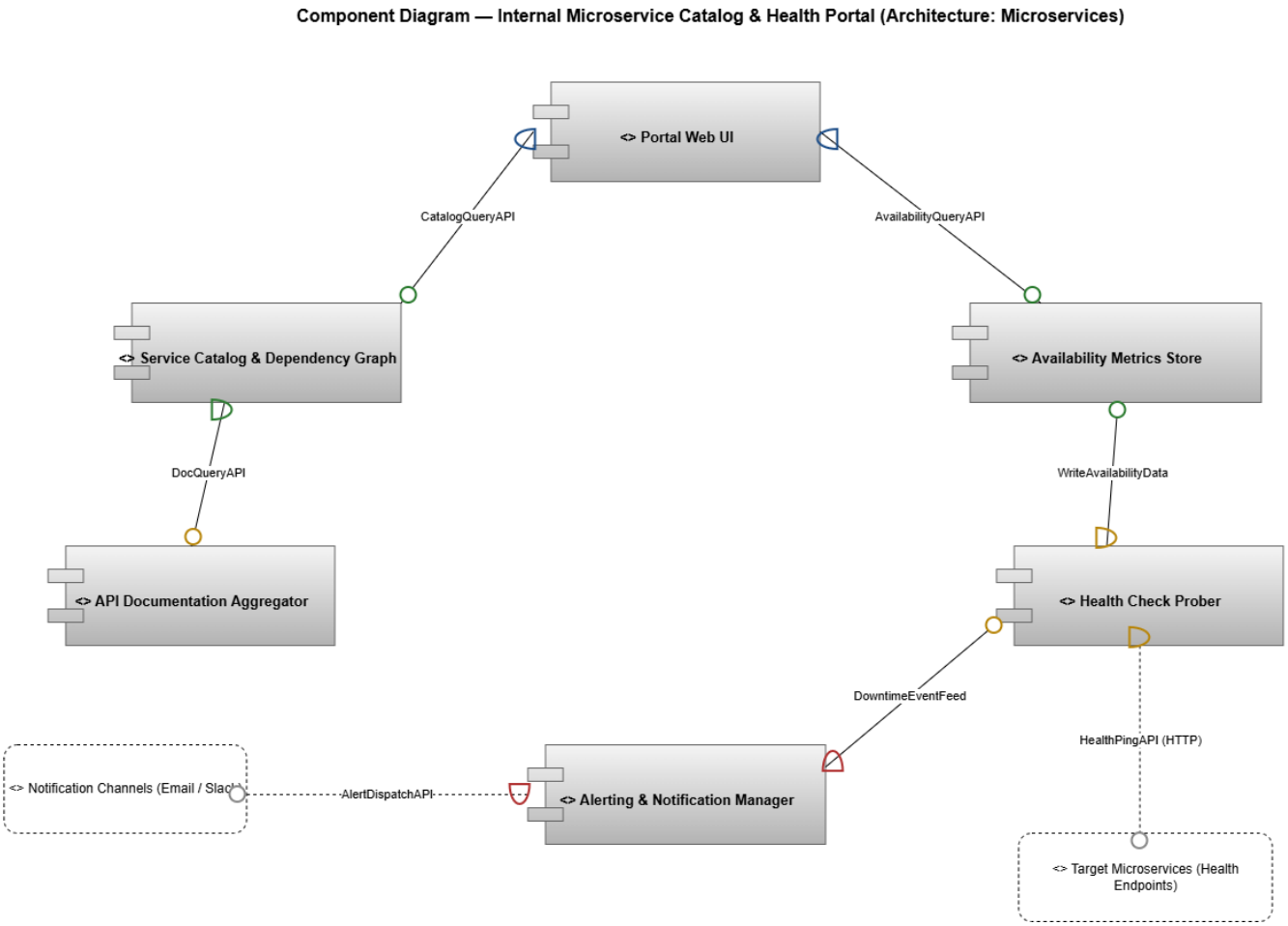
Architecture Selection

We chose a Microservices Architecture for the Internal Microservice Catalog & Health Portal system.

Architectural Choice

The system is decomposed into independently deployable components: Portal Web UI, Service Catalog & Dependency Graph, API Documentation Aggregator, Health Check Prober, Availability Metrics Store, and Alerting & Notification Manager — each communicating over well-defined interfaces rather than sharing a single process or database.

Component Diagram — Internal Microservice Catalog & Health Portal (Architecture: Microservices)



Two Scenario-Related Reasons

1. Independent scaling for divergent workloads — The Health Check Prober must ping up to 200 microservice health endpoints every 30 seconds (FR-001), a continuous, high-frequency background load that is completely different from the sporadic, human-driven traffic on the Portal Web UI and Dependency Graph viewer. Microservices let the Prober scale horizontally during peak monitoring load without over-provisioning the catalog-browsing components — something a Layered or Client-Server design would force to scale together.

2. Fault isolation across independently critical functions — Health checking, alert dispatch, and documentation aggregation have different reliability needs. If the API Documentation Aggregator stalls (e.g., a slow third-party OpenAPI fetch), that failure must not block downtime detection or graph rendering. Microservices contain a failure to a single service, whereas a monolithic Layered or Client-Server process would let it cascade across the whole portal.

Security Advantage

Each service is granted only the access it needs: only the Health Check Prober requires outbound network access to the monitored microservices' health endpoints, and only the Alerting & Notification Manager holds credentials for external channels (email/Slack). This limits the blast radius of a compromise, instead of concentrating broad network and credential access inside one monolithic process.

Performance Benefit

Separating the Health Check Prober and Availability Metrics Store from the Service Catalog means the 30-second polling workload does not compete with the interactive dependency-graph rendering required for up to 200 services (NFR-001). The Catalog can be optimized and cached for read-heavy graph queries while the Prober is optimized for write-heavy, high-frequency health ingestion, so neither workload degrades the other.